

A Comparative Analysis of Optimization Trajectories and Convergence Properties.

Ozair Faizan

ahmad.faizan@epfl.ch

Paul Tercier

paul.tercier@epfl.ch

Abstract—The choice of the optimization algorithm determines training efficiency and generalization of deep neural networks. While methods such as Stochastic Gradient Descent and Adam have become field standards, recent works such as Lion [1] and Muon [2] promise better convergence properties in practice. In this paper we present a comparative study of the optimization trajectories produced by Stochastic Gradient Descent, Adam, Lion and Muon applied to Multi-Layer Perceptron on the Fashion-MNIST dataset [3]. To this end, we use a linear interpolation method: for each discrete update step from θ_t to θ_{t+1} , we evaluate the loss function at $N = 100$ points along the connecting vector, allowing us to empirically verify convexity and smoothness properties along the trajectory.

Furthermore, we evaluate the convergence points m by testing the condition:

$$f(m) \leq f(x) + \nabla f(m)^\top (m - x),$$

for points x within a neighborhood $B_\epsilon(m)$.

We use constant learning rate found by grid-search and compare them with predicted theoretical values, all other hyperparameters are set to default.

I. INTRODUCTION

The choice of the optimizer is central to the process of training deep neural networks, influencing not only the speed of convergence but also the ability of the model to generalize. Even though there is a vast number of available optimizers that can be applied to train a deep neural network, [4] identified over one hundred unique algorithms, yet only two have dominated the scene: Stochastic Gradient Descent (SGD) [5] and Adam [6]. However, recent studies have refuted the common belief. According to [1], the Lion optimizer, which was developed based on symbolic search for update equations, is able to achieve similar levels of effectiveness without having problems with large amounts of memory usage. Furthermore, the Muon optimizer [2] employs Nesterov momentum using a spectral norm-based method.

Empirical comparisons typically report aggregate metrics such as final test accuracy or training loss curves. Less studied is the geometry of the optimization trajectory itself: whether successive parameter updates traverse locally convex regions of the loss landscape, how smooth the loss appears along each step, and whether convergence points satisfy local optimality conditions. These properties connect directly to quantities central to optimization theory such as Lipschitz smoothness, strong convexity, gradient dominance and may explain

behavioral differences between optimizers that aggregate metrics alone do not reveal.

II. MODELS AND METHODS

We fit a dense feedforward MLP model on the Fashion MNIST [3] dataset which includes 10 different classes for image classification. This dataset contains a total of 60,000 gray-scale images of size 28x28. Our model is composed of two layers, each having a width of 256 with activation ReLU, followed by a linear classification layer giving rise to a total parameter count of 270K. The training regime includes cross-entropy loss, batch size 128, and 5 epochs. No regularization or learning rate decay is employed for simplicity.

The following four optimization algorithms are compared. Stochastic gradient descent (SGD) updates its parameters using

$$\theta_{t+1} = \theta_t - \eta g_t.$$

Here, g_t represents the stochastic gradient. The Adam optimizer [6] computes the first- and second-moment averages of the gradient and performs bias-corrected adaptive steps per coordinate. The Lion optimizer [1] uses only the sign of a momentum-gradient sum as the update direction:

$$\theta_{t+1} = \theta_t - \eta \text{sign}(\beta_1 m_t + (1 - \beta_1) g_t).$$

The momentum is updated by

$$m_{t+1} = \beta_2 m_t + (1 - \beta_2) g_t.$$

The Muon optimizer [2] implements the Nesterov-type momentum in the direction of steepest descent in terms of the spectral norm and approximates it by performing the Newton-Schulz orthogonalization of the gradient matrix.

An independent grid search is carried out within the range of learning rates

$$\{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}\}.$$

for each optimizer, and the learning rate resulting in the minimum training loss at epoch 50 is chosen. All other hyperparameters remain unchanged from their published values (i.e., β_1, β_2 for Adam, Lion, Muon and the momentum coefficient for SGD).

For each optimizer, the parameter vectors θ_t and θ_{t+1} are stored at each training iteration. The loss function is calculated at the points along the straight line

$$\theta(\alpha) = (1 - \alpha) \theta_t + \alpha \theta_{t+1}, \quad \alpha \in \{0, 1/N, \dots, 1\},$$

where $N = 100$. This analysis allows us to check if the loss function along the update trajectory is convex, concave, or nonmonotone, thus providing a local approximation of the curvature of the loss landscape.

Let m denote the parameter vector at the end of training. We verify the first-order characterization of convexity:

$$f(m) \leq f(x) + \nabla f(m)^\top (m - x), \quad \forall x \in \mathcal{B}_\varepsilon(m), \quad (1)$$

by sampling $K = 50$ directions uniformly on the unit sphere and evaluating f at radius $\varepsilon \in \{10^{-3}, 10^{-2}, 10^{-1}\}$. A convergence point is flagged as satisfying the condition if the inequality holds for all sampled x at a given ε , providing a soft empirical measure of local convexity.

III. RESULTS

The selected learning rates and final values for both train loss and test accuracy after 5 epochs are shown in Table 1. All four optimizers show competitive test accuracy on Fashion-MNIST dataset. The highest test accuracy is achieved by both Adam 87.74% and Lion 87.81%, followed by Muon with 87.29% and SGD with 85.43%. Lion optimizer achieves an accuracy comparable to Adam with a learning rate that is 10x lower than the latter due to the sign-based update producing larger step sizes.

TABLE I
SELECTED LEARNING RATES AND FINAL PERFORMANCE AFTER 5 EPOCHS.

Optimizer	LR	Batch Size	Train Loss	Test Acc. (%)
SGD	10^{-2}	32	0.3741	85.43
Adam	10^{-3}	64	0.2859	87.74
Lion	10^{-4}	128	0.2866	87.81
Muon	10^{-3}	64	0.3921	87.29

IV. DISCUSSION

V. SUMMARY

REFERENCES

- [1] L. Chen, B. Liu, K. Liang, and Q. Liu, "Lion secretly solves constrained optimization: As lyapunov predicts," 2025. [Online]. Available: <https://arxiv.org/abs/2310.05898>
- [2] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein, "Muon: An optimizer for hidden layers in neural networks," 2024. [Online]. Available: <https://kellerjordan.github.io/posts/muon/>
- [3] H. Xiao, K. Rasul, and R. Vollgraf. (2017) Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms.
- [4] F. Schneider, F. Dangel, and P. Hennig, "Descending through a crowded valley — benchmarking deep learning optimizers," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021. [Online]. Available: <https://arxiv.org/abs/2007.01547>
- [5] H. Robbins and S. Monro, "A stochastic approximation method," *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951. [Online]. Available: <https://doi.org/10.1214/aoms/1177729586>
- [6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>